PNR Waitlist Confirmation Predictor - Project Guide

1. Project Description

Project Name: PNR Waitlist Ticket Confirmation Predictor
Tech Stack: Python, Streamlit, Scikit-Learn (Machine Learning), Docker
Model: Logistic Regression

Overview:
This project is a Machine Learning web application designed to predict whether an Indian Railways waitlisted ticket will get confirmed. It provides a polished, interactive user interface built with Streamlit.

Users can input various ticket details—such as Class of Travel, Quota, Train Type, Waitlist Position, and Days Before Journey—and the underlying Logistic Regression model calculates the probability of the ticket being confirmed.

2. Working of Nextcloud in this Project (Architecture & Role)

While the core ML application is built with Streamlit, **Nextcloud** plays a critical role as the secure cloud storage and versioning hub for the project's source code and model files.

How the Nextcloud Integration Works:

1. **WebDAV API:** Nextcloud operates on the WebDAV protocol, which allows our Python scripts to programmatically create folders and transfer files over HTTPS.
2. **Zero-Credential Authentication:** Instead of hardcoding a username and password (which is a major security risk), the project utilizes Nextcloud's **Public Share Tokens**. The system creates an anonymous "Drop Folder", and the token acts as the secure key.
3. **Automated Docker Uploads:** A dedicated Python script (`upload_to_nextcloud.py`) is executed inside an isolated Docker container. This script:
    - Scans the local project directory.
    - Automatically filters out unnecessary files (like `.git`, `venv`, and `__pycache__`).
    - Uses the `requests` library to establish a connection to Nextcloud.
    - Pushes all updated code files, models, and data directly into the Nextcloud cloud directory.

Why use Nextcloud here? It acts as a private, self-hosted alternative to GitHub/Google Drive, ensuring that all ML models (which can be large) and proprietary source code are securely backed up and centralized without relying on third-party public repositories.

---

3. How to Work on Your Project (The Workflow)

Since Nextcloud is acting as your secure backup/storage for this project, here is the complete

end-to-end workflow on how to develop, run, and upload your project.

Step 1: Running the Application Locally

To actually see and use your application, you need to run it on your own computer. You have a `Dockerfile` specifically designed for this.

Open your terminal in your project folder (`D:\mlpro`) and run:

```
# Build the application image
docker build -t my-ml-app .

# Start the application on your computer
docker run -p 8501:8501 my-ml-app
```

*Open your web browser and go to **http://localhost:8501** to view the live application.*

Step 2: Making Changes to the Code

When you want to improve the model or change the app's look:

1. Open the project files in your code editor (like VS Code).
2. To change the frontend, edit `app/app.py`.
3. To retrain the model, modify the scripts in the `src/` directory.
4. Save your files and refresh your browser to see the changes if running locally.

Step 3: Uploading (Backing up) to Nextcloud

Once you have made changes and want to back up your code to the cloud, use your dedicated Nextcloud uploader.

Because we set up the "Zero-Credentials" Public Share token in the `.env` file, uploading is as simple as running a single Docker command.

Run this in your terminal (`D:\mlpro`):

```
docker run --rm `
  -v "${PWD}:/app" `
  -w /app `
  --env-file .env `
  python:3.12-slim `
  bash -c "pip install requests && python upload_to_nextcloud.py"
```

This spins up a temporary container, packages your updated code, sends it to your Nextcloud share link, and deletes itself.

---

3. How to Save This as a PDF

To save this guide as a PDF:

1. If you are reading this in the Artifact window on the right side of the screen, you can copy the text into Microsoft Word or Google Docs and click **File** > **Download** > **PDF Document (.pdf)**.

2. Alternatively, you can use your browser's Print feature (press `Ctrl + P`) right now and change the Destination to "**Save as PDF**".